MbW Lathe — Raspberry Pi 4B Hardware Wiring Guide

> ***Target hardware:*** *Raspberry Pi 4B · ClearPath SDSK servos · CUI AMT103-V encoders · AutoTech C3 spindle sensor*
> ***GPIO numbering:*** *BCM throughout*
> ***Signal voltage:*** *RPi GPIO = 3.3 V logic — level shifters* ***mandatory*** *for 5 V devices*

Table of Contents

---

1. Component List

All items are available as off-the-shelf modules from common electronics retailers.

1.1 Core Processing

#	Component	Model / Spec	Qty	Source
1	Raspberry Pi 4B (4 GB)	RPi 4 Model B	1	raspberrypi.com, Adafruit, PiShop
2	microSD Card (32 GB A1)	Samsung Endurance Pro	1	Amazon
3	RPi USB-C Power Supply	Official 5.1 V / 3 A	1	raspberrypi.com
4	7" HDMI Capacitive Touchscreen	Elecrow 800×480 or equiv.	1	Amazon, Elecrow

1.2 Motion & Sensing

#	Component	Model / Spec	Qty	Notes
5	Handwheel Encoder	CUI AMT103-V	2	2000 CPR, 3.3 V compatible
6	Encoder Cable	CUI 435-1FT	2	5-pin MTA-100 connector
7	ClearPath Servo (Z-Axis)	Teknic SDSK	1	Existing — 800 counts/rev in MSP
8	ClearPath Servo (X-Axis)	Teknic SDSK	1	Same as Z
9	Servo Signal Cable	Teknic CPM-CABLE-CTRL-MU120	2	8-pin Molex Mini-Fit, one per motor
10	Servo Power Cable	Teknic CPM-CABLE-PWR-MS120	2	One per motor

11	Spindle Index Card + Sensor	AutoTech C3	1	5 V TTL output, single-hole optical

1.3 Signal Conditioning

#	Component	Model / Spec	Qty	Notes
12	Bidirectional Level Shifter	TXS0108E Module (8-channel)	2	One per servo axis (3.3 V → 5 V). Uses 3 of 8 channels per module.
13	ADC Module	Adafruit ADS1115	1	16-bit, I2C, 3.3 V
14	10 kΩ Resistor (1/4 W)	Through-hole	10	Pull-ups, pull-downs, V-divider
15	20 kΩ Resistor (1/4 W)	Through-hole	2	Spindle voltage divider lower leg

1.4 User Inputs

> *These are the operator controls read directly by the RPi GPIO. All are defined in `config.py`.*

#	Component	Model / Spec	Qty	Notes
16	Potentiometer (10 kΩ linear)	Bourns 3547S or equiv.	1	Feed-rate knob → ADS1115 A0

17	Momentary Push Button (NO)	2-pin panel mount	3	Memory / Zero / Units (GPIO 26/20/21)
18	Half-Nut Lever Microswitch	Omron D2HW-C213MR or equiv.	1	Lever-actuated NO contact (GPIO 4)
19	Limit Switch (NO)	Generic panel mount	4	Z+/Z–/X+/X– (GPIO 16/7/8/11)

1.5 Safety

> *Hard-wire E-Stop path. The relay is controlled directly by the physical button — no GPIO line required from the RPi.*

#	Component	Model / Spec	Qty	Notes
20	E-Stop Button (40 mm mushroom)	Latching NC/NO combo	1	ABB / Schneider / Idec preferred
21	2-Channel 5 V Relay Module	Optocoupler-isolated, JD-VCC type	1	Hard-cuts ClearPath ENABLE line

1.6 Power & Wiring

#	Component	Model / Spec	Qty	Notes
22	70 V DC Power Supply	Generic, ≥7 A	1	Servo bus power
23	DC-DC Buck Converter Module	LM2596 adjustable or XL4016	1	Set to 5 V — for C3 sensor + relay

24	Terminal Blocks 3-pos	Phoenix Contact 3044513	9	DIN rail mount
25	35 mm DIN Rail (300 mm)	Generic	1	Terminal block mounting
26	Small Breadboard or Strip-board	400-point	1	Voltage divider + signal conditioning
27	Hookup Wire (22 AWG stranded)	Assorted colours	1 roll	General wiring
28	Heat-shrink Tubing	Assorted	1 pack	Insulation

---

2. Power Rail Extraction

The RPi GPIO header exposes two usable power rails. Understand the current budgets **before** tapping them.

2.1 RPi GPIO Header Power Pins

```
RPi GPIO 40-pin Header — Power pins ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ [1] 3.3V ■■■ [2] 5V ■ ← Both usable ■ [3] SDA ■■■ [4] 5V ■ ← Extra 5V tap ■
[5] SCL ■■■ [6] GND ■ ■ ... ■ ■ [17] 3.3V ■■■ [18] GPIO24 ■ ← Second 3.3V tap ■
[25] GND ■■■ [26] GPIO7 ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ 3.3 V rail
(Pins 1 and 17) Source: onboard LDO regulator Budget: 500 mA max Used for: Z
Encoder (~20 mA), X Encoder (~20 mA), ADS1115 (<1 mA), Level Shifter VCCA (×2, ~2
mA each) Total load in this project: ~50 mA ✓ well within budget 5 V rail (Pins 2
and 4) Source: direct from USB-C input rail (unfused on header) Budget: ~1.5–2 A
with official 3 A PSU (RPi draws ~600 mA at idle) Used for: Level Shifter VCCB (×2,
~5 mA each) ■ Do NOT power the AutoTech C3 or relay module from this rail — use the
external buck module instead
```

2.2 External 5 V from 70 V Rail (Buck Converter)

Use a buck converter module to generate a clean isolated 5 V from the servo supply. This offloads current from the RPi's USB-C PSU and cleanly powers high-frequency sensor circuits.

```
70 V DC Bus ■■■■■ IN+ ■■■■■■■■■■■■■■■■■■■■■■ OUT+ ■■■■■ 5 V (to C3, relay module) ■ LM2596 Buck ■ IN- ■■■■■■■■■■■■■■■■■■■■■■■■■■ OUT- ■■■■■ GND (common with RPi GND) Setup procedure: 1. Connect input only. Power on 70 V PSU. 2. Adjust trim potentiometer until VOUT = 5.00 V (measure with multimeter). 3. Power off. Connect loads (C3, relay module). 4. Verify output again under load — should remain 5.0 V ± 0.1 V.
```

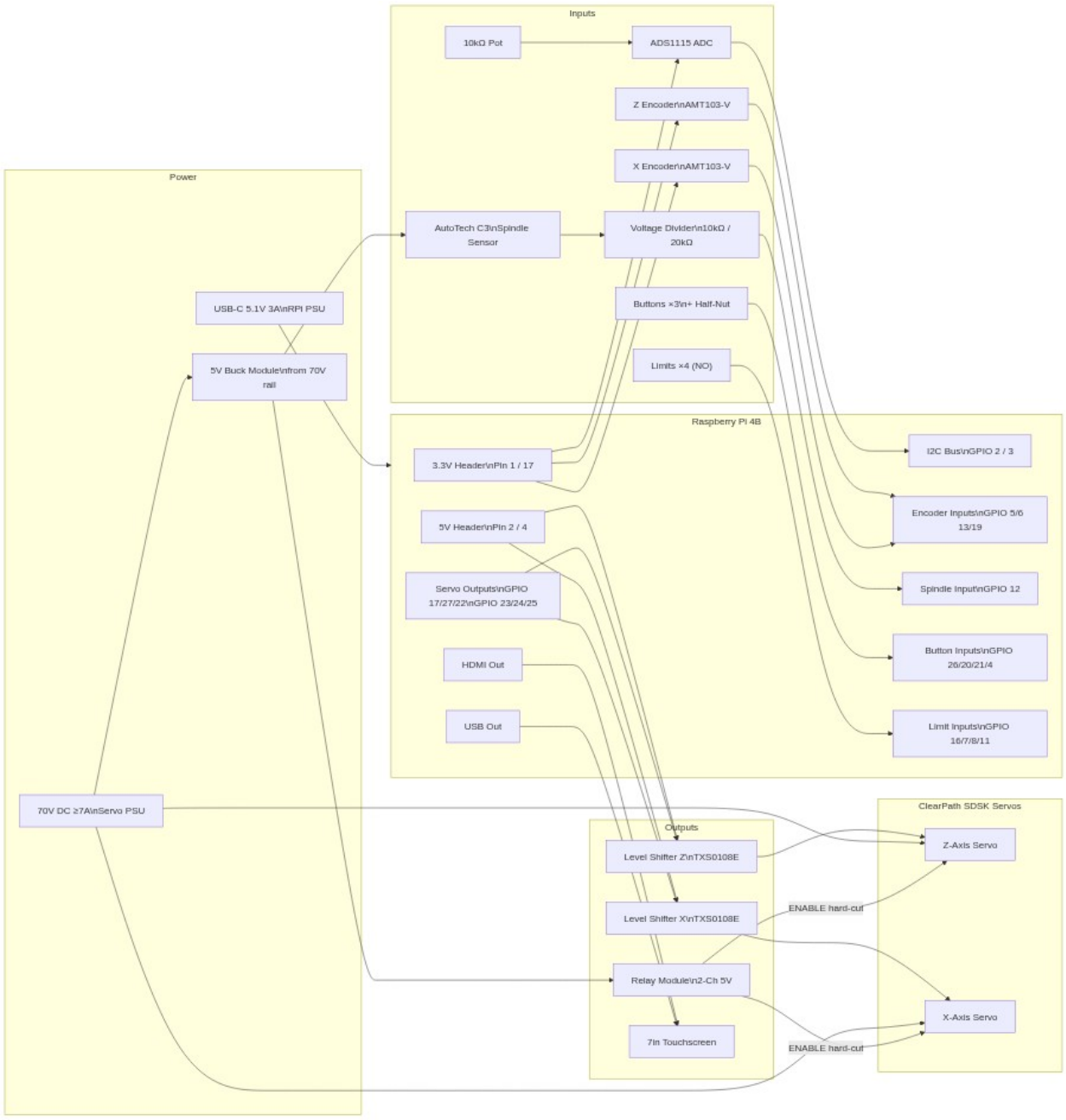
> ***Ground rule:*** *All GNDs in this system must be tied to a single common ground bus — RPi GND, buck module GND, 70 V PSU GND. Floating grounds cause erratic signals.*

2.3 Power Budget Summary

Consumer	Supply	Est. Current
RPi 4B (idle)	USB-C 5.1 V / 3 A PSU	~600 mA
7" Touchscreen (via USB)	USB from RPi	~500 mA
Z Encoder (AMT103-V)	3.3 V header	~20 mA
X Encoder (AMT103-V)	3.3 V header	~20 mA
ADS1115 ADC	3.3 V header	<1 mA
Level Shifter Z — VCCA	3.3 V header	~2 mA
Level Shifter X — VCCA	3.3 V header	~2 mA
Level Shifter Z — VCCB	5 V (GPIO Pin 2)	~2 mA
Level Shifter X — VCCB	5 V (GPIO Pin 2)	~2 mA
AutoTech C3 sensor	5 V (buck module)	~50 mA
Relay module (2-ch)	5 V (buck module)	~80 mA
ClearPath Z + X servos	70 V DC	≤7 A peak

---

3. System Overview Diagram



---

4. Step-by-Step Connections

> ***Safety rule:*** *Power off* ***all*** *supplies before making or changing any wiring. Only power up during the verification steps.*

Step 1 — Power Setup

Parts needed: RPi USB-C PSU, 70 V DC PSU, LM2596 buck module, mains power strip with switch, multimeter.

1a. Adjust Buck Module (do this before connecting any load)

```
1. Connect buck IN+ to 70 V PSU terminal + 2. Connect buck IN− to 70 V PSU terminal
− (GND) 3. Power ON the 70 V PSU (nothing else connected) 4. Measure VOUT with
multimeter — trim potentiometer until VOUT = 5.00 V 5. Power OFF. Wait 30 s for
capacitors to discharge.
```

1b. Common Ground Bus

All grounds must be connected together. Use a 3-position terminal block on DIN rail as the GND bus.

```
70 V PSU GND ■■■ Buck GND ■■■■■■■■■■■■■ Common GND Bus (terminal block) ■■■■ RPi
GND (Pin 6 / 9 / 14) RPi PSU GND ■■■■
```

Verification — Step 1

Test	Expected Result	Tool
Buck VOUT (no load)	5.00 V ± 0.05 V	Multimeter
70 V PSU output	68–72 V	Multimeter
GND continuity bus-to-bus	< 1 Ω	Continuity mode

Step 2 — RPi + Touchscreen

Parts needed: Raspberry Pi 4B, microSD with OS flashed, USB-C PSU, 7" HDMI touchscreen.

Wiring

```
Raspberry Pi 4B 7" Touchscreen ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■ Micro-HDMI port (use port 0) ■■■■ HDMI input USB-A port
■■■■■■■■■ USB (touch input) USB-C port ■■■■■■■■■ USB-C PSU (5.1 V / 3 A)
```

> *Use a micro-HDMI to full-HDMI cable. RPi 4 has **micro-HDMI** (not mini-HDMI).*

Verification — Step 2

Test	Expected Result

RPi powers on	Red LED on, green LED activity
Touchscreen displays RPi desktop	800×480, no artefacts
Touch input responds	Cursor moves, tap registers

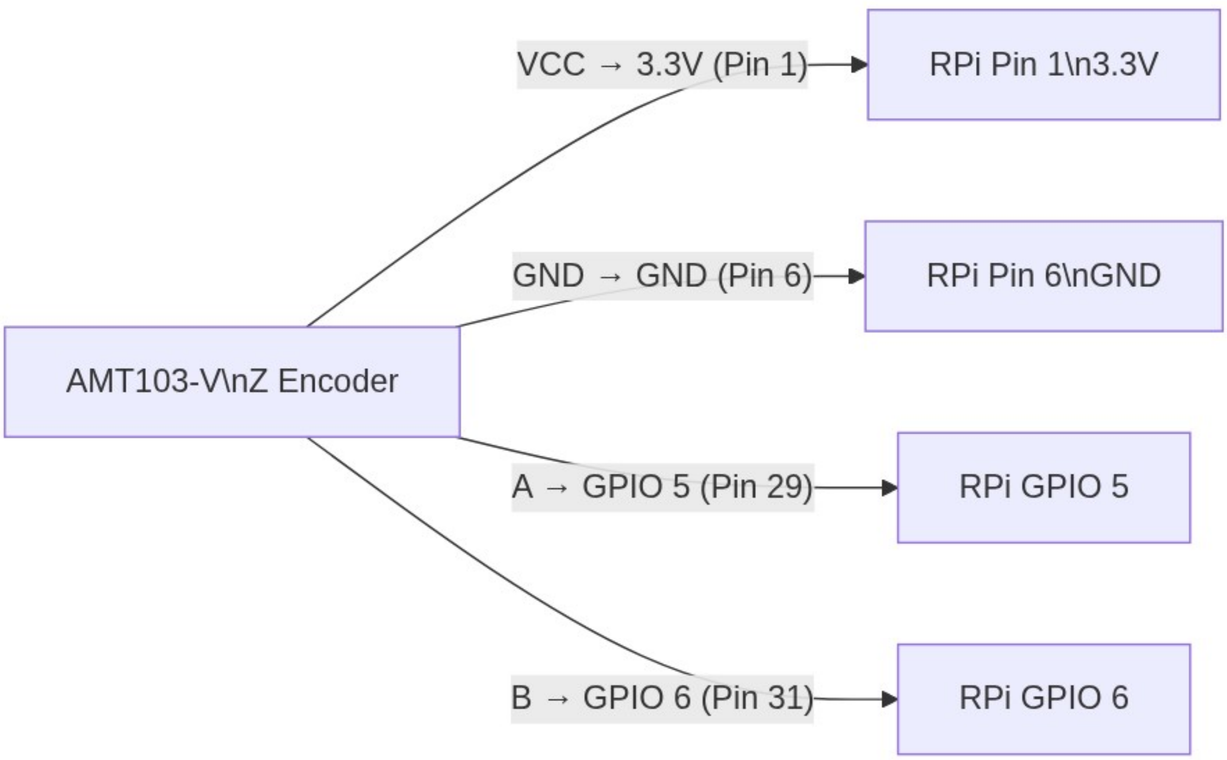
Step 3 — Z-Axis Encoder (CUI AMT103-V)

Parts needed: CUI AMT103-V encoder, CUI 435-1FT cable.

The AMT103-V operates at 3.3 V and is directly compatible with RPi GPIO — no level shifter needed.

Wiring

```
AMT103-V → Raspberry Pi 4B ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ Pin 1 (VCC) ■■■■ 3.3 V (Pin 1) Pin 2 (GND) ■■■■ GND (Pin 6) Pin 5 (A) ■■■■ GPIO 5 (Pin 29) Pin 6 (B) ■■■■ GPIO 6 (Pin 31) Pin 3/4 (IDX) ■■■■ Not connected
```



Verification — Step 3

```
# Run on RPi — test Z encoder (requires pigpiod running) import pigpio, time pi = pigpio.pi() pi.set_mode(5, pigpio.INPUT) pi.set_mode(6, pigpio.INPUT) pi.set_pull_up_down(5, pigpio.PUD_UP) pi.set_pull_up_down(6, pigpio.PUD_UP)
```

```
print("Rotate Z handwheel. GPIO 5 and 6 should toggle.") for _ in range(30):
print(f" A={pi.read(5)} B={pi.read(6)}", end='\r') time.sleep(0.1) pi.stop()
```

Test	Expected Result
GPIO 5 reads 1 at rest	Internal pull-up active
GPIO 5 and 6 toggle on rotation	Quadrature signal present
A leads B on CW rotation	Phase relationship correct

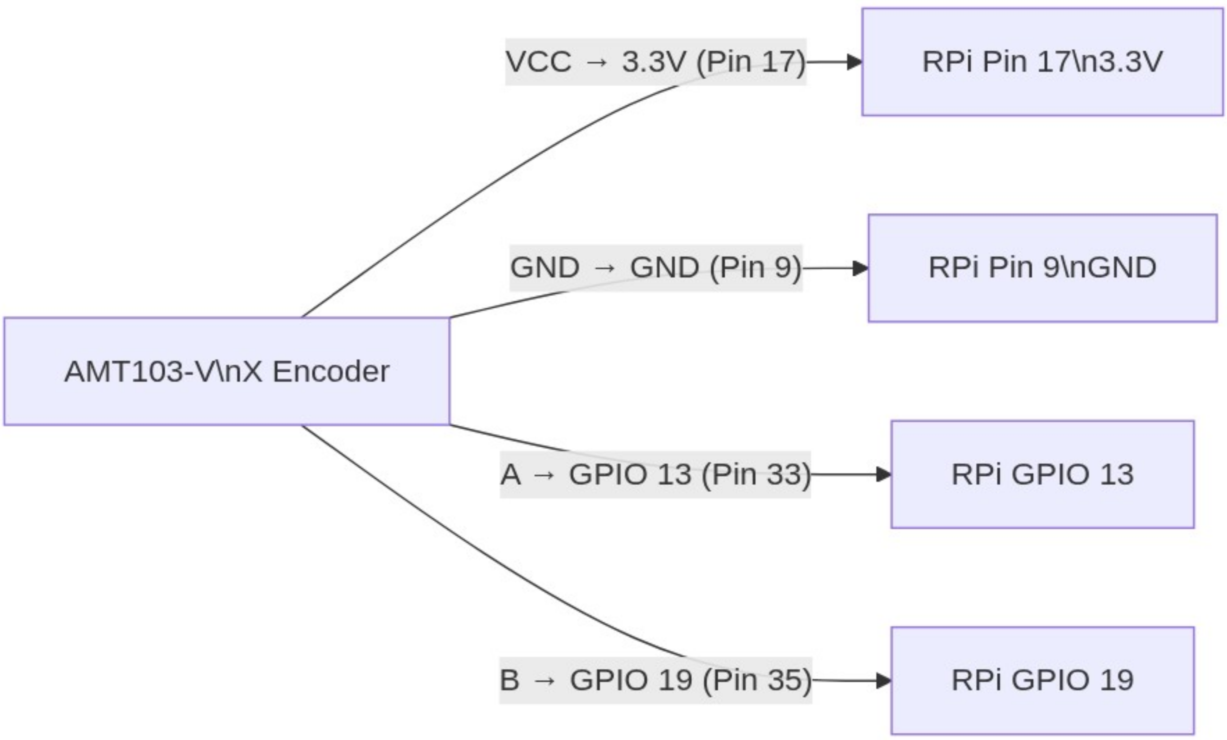
Step 4 — X-Axis Encoder (CUI AMT103-V)

Parts needed: Second CUI AMT103-V encoder, CUI 435-1FT cable.

Identical wiring to Step 3 — uses GPIO 13 (A) and GPIO 19 (B).

Wiring

```
AMT103-V → Raspberry Pi 4B ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ Pin 1
(VCC) ■■■■ 3.3 V (Pin 17) Pin 2 (GND) ■■■■ GND (Pin 9) Pin 5 (A) ■■■■ GPIO 13
(Pin 33) Pin 6 (B) ■■■■ GPIO 19 (Pin 35) Pin 3/4 (IDX) ■■■■ Not connected
```



Verification — Step 4

```
import pigpio, time pi = pigpio.pi() pi.set_mode(13, pigpio.INPUT) pi.set_mode(19, pigpio.INPUT) pi.set_pull_up_down(13, pigpio.PUD_UP) pi.set_pull_up_down(19, pigpio.PUD_UP) print("Rotate X handwheel. GPIO 13 and 19 should toggle.") for _ in range(30): print(f" A={pi.read(13)} B={pi.read(19)}", end='\r') time.sleep(0.1) pi.stop()
```

Step 5 — Spindle Sensor + Voltage Divider

Parts needed: AutoTech C3 card + optical sensor, 10 kΩ resistor (R1), 20 kΩ resistor (R2), small breadboard.

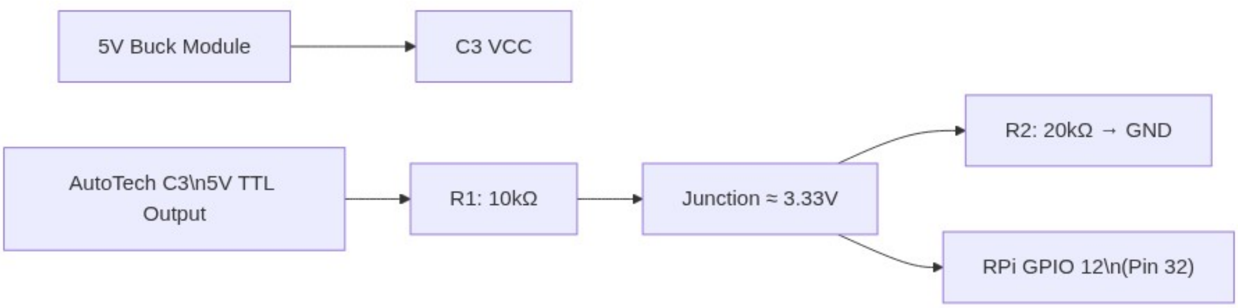
> ■ ***Mandatory:*** *The C3 outputs **5 V TTL**. RPi GPIO maximum input is **3.3 V**. A voltage divider is required.*

Voltage Divider Circuit

```
C3 Signal Output (5 V TTL) ■ 10 kΩ ← R1 ■ ■■■■■■■■■■■■■■■■ GPIO 12 (Pin 32) on RPi ■ V_out = 5V × 20k / (10k + 20k) = 3.33 V ✓ 20 kΩ ← R2 ■ GND
```

Full C3 Wiring

```
AutoTech C3 → Destination ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ VCC (+5 V) ■■■■ 5 V (buck module output) GND ■■■■ Common GND Signal Out ■■■■ R1 (10 kΩ) → junction → R2 (20 kΩ) → GND ■■■ GPIO 12 (Pin 32)
```



Verification — Step 5

```
import pigpio, time pi = pigpio.pi() pulse_count = [0] def on_pulse(gpio, level, tick): pulse_count[0] += 1 pi.set_mode(12, pigpio.INPUT) pi.callback(12, pigpio.RISING_EDGE, on_pulse) print("Rotate spindle slowly by hand. Count increments once per revolution.") start = time.time() while time.time() - start < 15: print(f" Pulses: {pulse_count[0]}", end='\r') time.sleep(0.1) pi.stop()
```

Test	Expected Result	Tool

Voltage at junction node	3.30–3.36 V	Multimeter
GPIO 12 level between pulses	0 (low)	Script above
Pulses per spindle revolution	Exactly 1	Script above

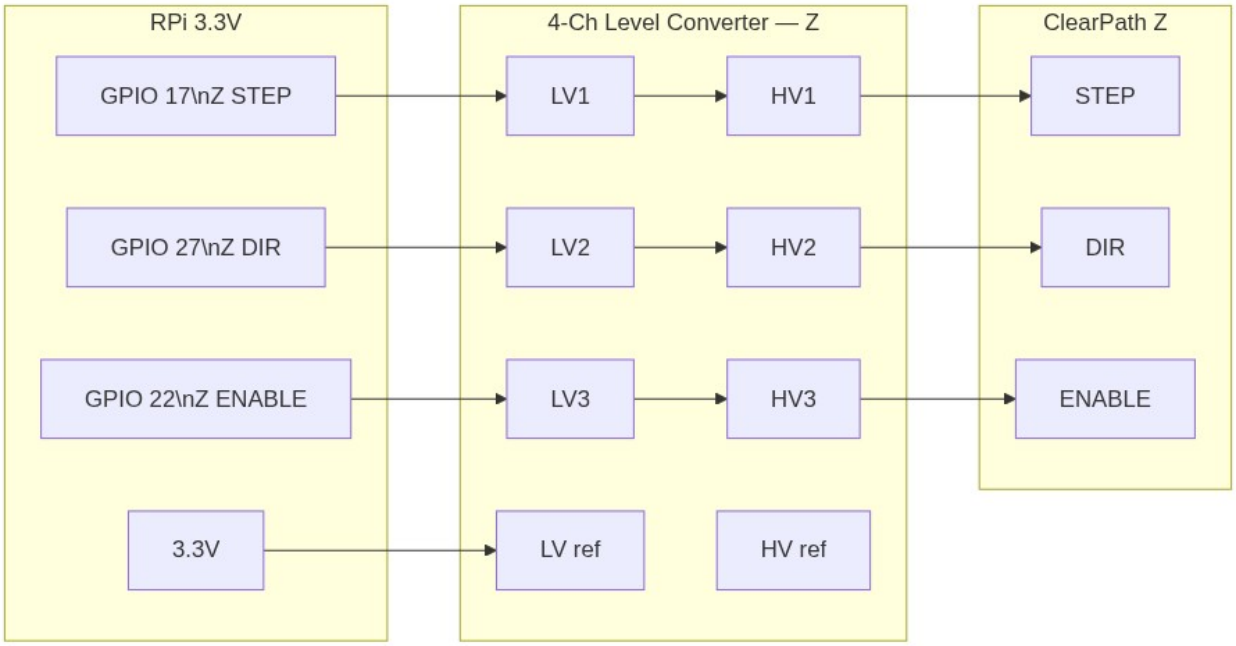
Step 6 — Z-Axis Level Converter + ClearPath Z Servo

Parts needed: 4-Channel Logic Level Converter module ×1, ClearPath Z signal cable (CPM-CABLE-CTRL-MU120).

> ■ ***Mandatory:*** *RPi outputs 3.3 V logic; ClearPath STEP / DIR / ENABLE require 5 V TTL. The level converter shifts all three signals. Only 3 of the 4 channels are used — 1 channel spare.*

Level Converter Wiring — Z Axis

```
RPi (3.3 V side — LV) 4-Ch Level Converter ClearPath Z (5 V side — HV)
■■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ GPIO 17 (Z STEP) ■■■■■■■ LV1 HV1 ■■■■■ STEP
input GPIO 27 (Z DIR) ■■■■■■■ LV2 HV2 ■■■■■ DIR input GPIO 22 (Z EN) ■■■■■■■ LV3
HV3 ■■■■■ ENABLE input (via relay NC, Step 13) 3.3 V (Pin 1) ■■■■■■■ LV (ref) HV
■■■■■ 5 V (GPIO Pin 2) GND ■■■■■■■ GND GND
```



ClearPath SDSK Signal Connector (8-pin Molex Mini-Fit)

```
ClearPath Pin → Destination
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ STEP (A) →
Level Converter HV1 (Z output) DIR (B) → Level Converter HV2 ENABLE (E) → Relay
NC1 → Level Converter HV3 (see Step 13) HLFB (H) → Not used (optional fault
feedback) GND → Common GND +5 V / VCC → 5 V (GPIO Pin 2) HV+, HV− → 70 V DC bus
(Step 8)
```

> ***ENABLE logic:*** *HIGH = servo enabled, LOW = disabled. The relay in Step 13 hard-cuts this line on E-Stop.*

Verification — Step 6

```
# Signal test — motor power OFF, signal cable connected import pigpio, time pi =
pigpio.pi() for pin in [17, 27, 22]: pi.set_mode(pin, pigpio.OUTPUT) pi.write(pin,
0) print("Measure TXS0108E B2 (Z DIR) with multimeter:") pi.write(27, 0); print("
GPIO 27 = LOW → HV2 should read ~0 V"); time.sleep(2) pi.write(27, 1); print(" GPIO
27 = HIGH → HV2 should read ~5 V"); time.sleep(2) pi.write(27, 0) pi.stop()
```

Test	Expected Result	Tool
VCCA on TXS0108E	3.3 V	Multimeter
VCCB on TXS0108E	5.0 V	Multimeter
B1 / B2 / B3 when RPi drives HIGH	4.8–5.0 V	Multimeter
B1 / B2 / B3 when RPi drives LOW	0 V	Multimeter

Step 7 — X-Axis Level Shifter + ClearPath X Servo

Parts needed: TXS0108E module ×1, ClearPath X signal cable.

Identical to Step 6 — use GPIO 23 (STEP), 24 (DIR), 25 (ENABLE) on the second TXS0108E module.

Level Shifter Wiring — X Axis

```
RPi (3.3 V side) TXS0108E Module ClearPath X (5 V side)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■ GPIO 23 (X STEP) ■■■■■■■ A1 B1 ■■■■ STEP input GPIO
24 (X DIR) ■■■■■■■ A2 B2 ■■■■ DIR input GPIO 25 (X EN) ■■■■■■■ A3 B3 ■■■■ ENABLE
```

```
input (via relay NC, Step 13) 3.3 V (Pin 17) ■■■■■■■ VCCA VCCB ■■■■ 5 V (GPIO Pin 4) GND ■■■■■■■ GND GND OE pin ■■■■■■■ Tie to VCCA (always enabled)
```

Verification — Step 7

Same test as Step 6 — substitute GPIO 23 / 24 / 25 and the second TXS0108E module.

Step 8 — ClearPath 70 V Motor Power

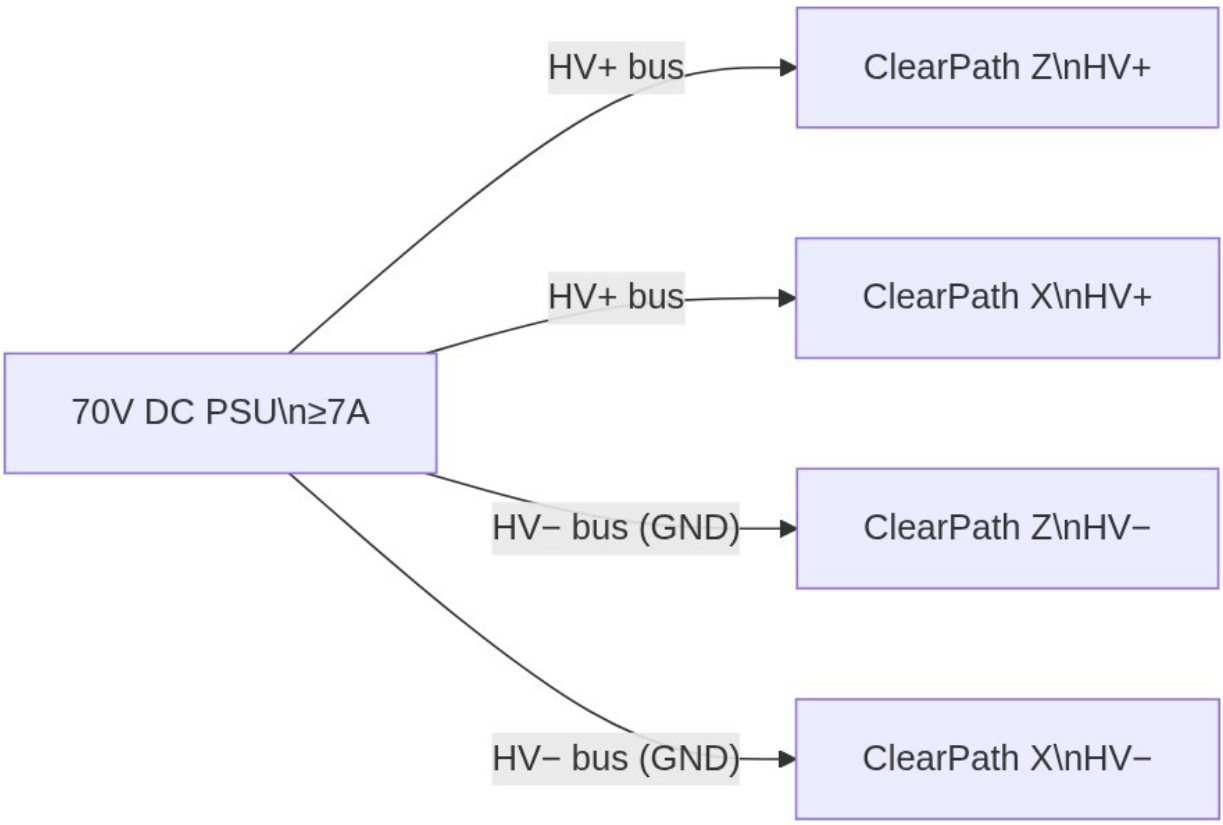
Parts needed: 70 V DC PSU (already set up in Step 1), Teknic CPM-CABLE-PWR-MS120 (×2).

> ■ ***High voltage.*** *The 70 V bus is hazardous. Verify all connections are insulated before powering up. Motors must be mechanically secured.*

Wiring

```
70 V DC PSU → ClearPath Motors ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ PSU V+ ■■■■■■■■■■■■■■■■■■■■ ClearPath Z HV+ terminal ■■■■■■■■■■■■■■■■■■ ClearPath X HV+ terminal PSU GND ■■■■■■■■■■■■■■■■■■■■ ClearPath Z HV− terminal ■■■■■■■■■■■■■■■■■■ ClearPath X HV− terminal Both motors share the same 70 V bus (parallel connection).
```

> ***Wire gauge:*** *Use minimum 16 AWG for the 70 V bus wires.*



Verification — Step 8

Test	Expected Result	Tool
70 V PSU output under no load	68–72 V	Multimeter
No motor movement at power-up	Correct — motors are inactive until ENABLE + STEP are driven	Visual
ClearPath HLFB LED (after ENABLE assertion)	Solid green = ready	Visual

---

Step 9 — ADS1115 ADC + Potentiometer

Parts needed: Adafruit ADS1115 module, 10 kΩ linear potentiometer.

The RPi has no onboard ADC. The ADS1115 provides 16-bit analog-to-digital conversion over I2C for the feed-rate potentiometer.

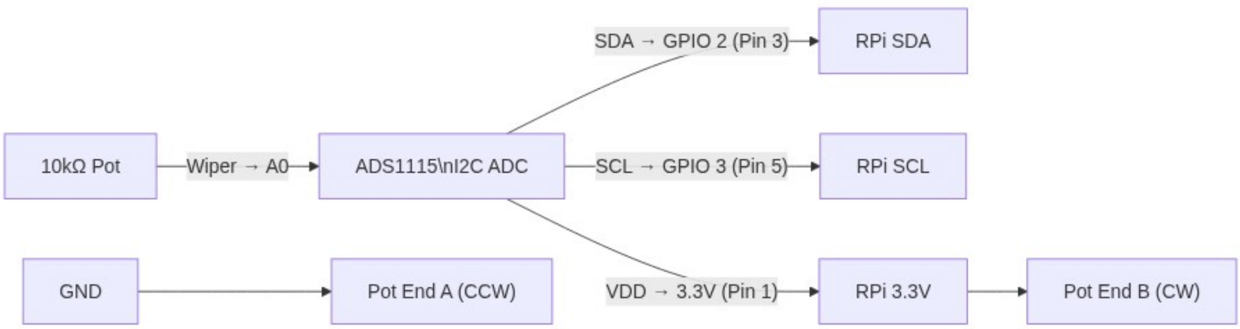
I2C Wiring (ADS1115)

```
ADS1115 Module → Raspberry Pi 4B ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ VDD ■■■■ 3.3 V (Pin 1) GND ■■■■ GND (Pin 6) SDA ■■■■ GPIO 2 / SDA (Pin 3) SCL ■■■■ GPIO 3 / SCL (Pin 5) ADDR ■■■■ GND → sets I2C address to 0x48 ALRT ■■■■ Not connected
```

Potentiometer Wiring

```
Potentiometer → Destination ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ End A (CCW stop) ■■■■ GND Wiper (centre) ■■■■ ADS1115 A0 input End B (CW stop) ■■■■ 3.3 V
```

> *Use **3.3 V** (not 5 V) to power the pot ends. The ADS1115 A0 input maximum is VDD = 3.3 V.*



Verification — Step 9

```
# Enable I2C on RPi (once) sudo raspi-config # Interface Options → I2C → Enable sudo reboot # Scan for ADS1115 on the I2C bus sudo apt install -y i2c-tools i2cdetect -y 1 # Expected: "48" appears at address 0x48
```

```
import board, busio from adafruit_ads1x15.ads1115 import ADS1115 from adafruit_ads1x15.analog_in import AnalogIn i2c = busio.I2C(board.SCL, board.SDA) ads = ADS1115(i2c) chan = AnalogIn(ads, 0) # A0 channel print(f"Voltage: {chan.voltage:.3f} V") # Rotate pot fully CW → ~3.3 V; fully CCW → ~0.0 V
```

Test	Expected Result
`i2cdetect` output	Address 0x48 visible
Pot at minimum (CCW)	~0.0 V
Pot at maximum (CW)	~3.3 V
Pot at midpoint	~1.65 V

Step 10 — Push Buttons (×3)

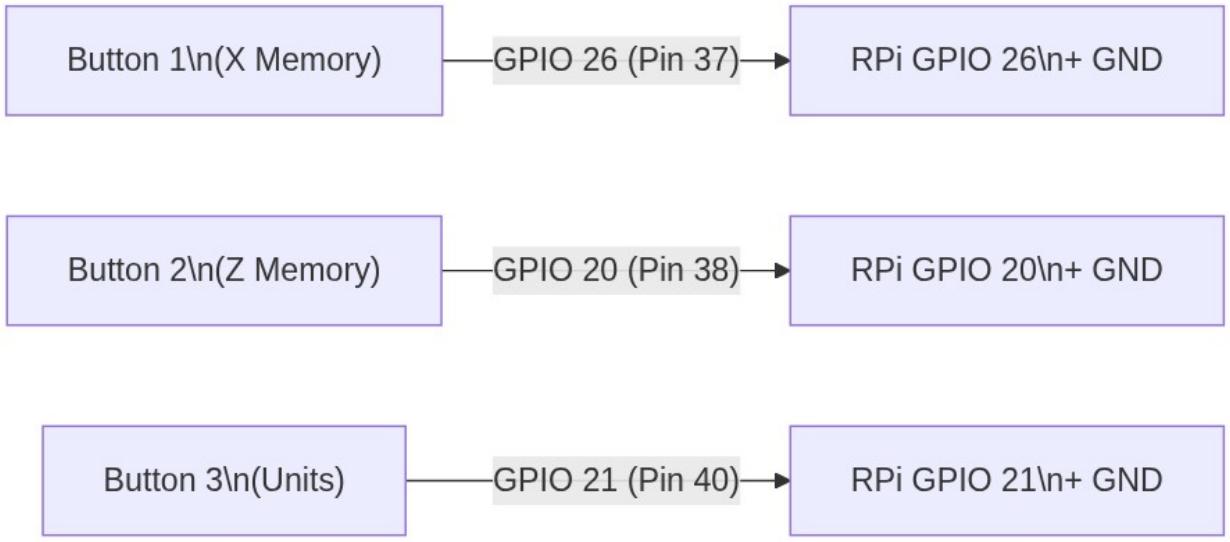
Parts needed: Momentary push buttons ×3 (NO, 2-pin).

GPIO internal pull-up is used. Buttons are **active LOW** — pressed = 0 V, released = 3.3 V.

Wiring (per button)

```
GPIO pin ■■■■■■■■ Button terminal A Button terminal B ■■■■ GND No external resistor needed — RPi internal pull-up is enabled in software.
```

Button	Function	GPIO (BCM)	Physical Pin
Button 1	X Memory / Zero	GPIO 26	Pin 37
Button 2	Z Memory / Zero	GPIO 20	Pin 38
Button 3	Units / Stop	GPIO 21	Pin 40



Verification — Step 10

```
import pigpio, time pi = pigpio.pi() for pin in [26, 20, 21]: pi.set_mode(pin, pigpio.INPUT) pi.set_pull_up_down(pin, pigpio.PUD_UP) print("Press each button. Its value should drop to 0.") for _ in range(40): print(f" BTN1={pi.read(26)} BTN2={pi.read(20)} BTN3={pi.read(21)}", end='\r') time.sleep(0.2) pi.stop()
```

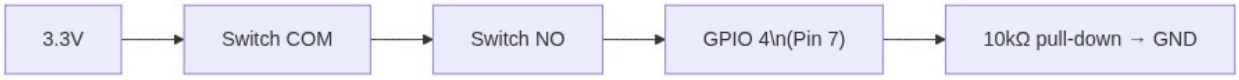
Step 11 — Half-Nut Lever Switch

Parts needed: Lever microswitch (Omron D2HW-C213MR or equivalent), 10 kΩ resistor (pull-down).

This switch is **active HIGH** — lever engaged = 3.3 V, lever released = 0 V.

Wiring

```
3.3 V ■■■■■■■■■■■■ Switch COM (common terminal) Switch NO
■■■■■■■■■■■■■■■■■■■■■■■■■ GPIO 4 (Pin 7) ■ 10 kΩ ← pull-down to GND ■ GND
```



Verification — Step 11

```
import pigpio, time pi = pigpio.pi() pi.set_mode(4, pigpio.INPUT)
pi.set_pull_up_down(4, pigpio.PUD_DOWN) print("Engage half-nut lever. GPIO 4 should
read 1.") for _ in range(25): print(f" Half-Nut = {pi.read(4)}", end='\r')
time.sleep(0.2) pi.stop()
```

Step 12 — Limit Switches (×4)

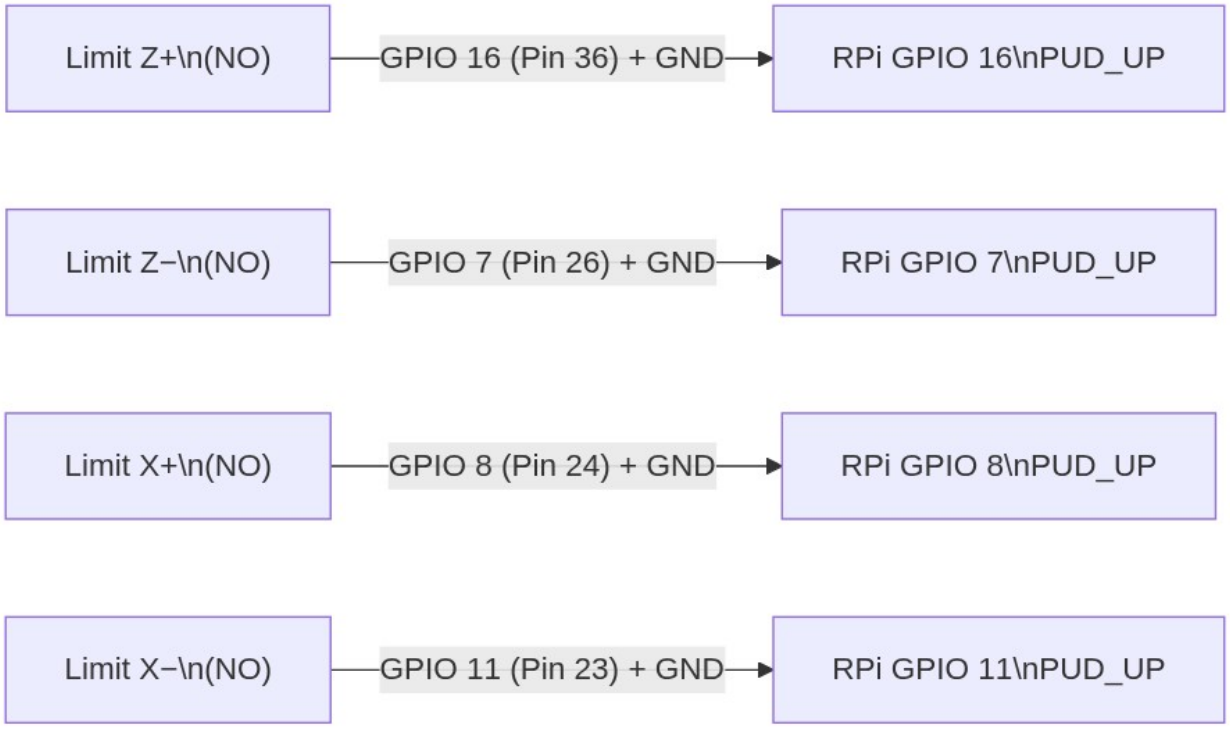
Parts needed: Normally Open (NO) panel-mount switches ×4.

GPIO internal pull-up is used. Switches are **active LOW** — triggered = 0 V.

Wiring (per limit switch)

```
GPIO pin ■■■■■■■■ Switch terminal A Switch terminal B ■■■■ GND No external
resistor needed — RPi internal pull-up is enabled in software.
```

Axis	Position	GPIO (BCM)	Physical Pin
Z+	Far travel end	GPIO 16	Pin 36
Z–	Home / near end	GPIO 7	Pin 26
X+	Far travel end	GPIO 8	Pin 24
X–	Home / near end	GPIO 11	Pin 23



Verification — Step 12

```
import pigpio, time pi = pigpio.pi() for pin in [16, 7, 8, 11]: pi.set_mode(pin, pigpio.INPUT) pi.set_pull_up_down(pin, pigpio.PUD_UP) print("Trigger each limit switch. Its value should drop to 0.") for _ in range(40): print(f" Z+={pi.read(16)} Z-={pi.read(7)} X+={pi.read(8)} X-={pi.read(11)}", end='\r') time.sleep(0.2) pi.stop()
```

Step 13 — E-Stop + Relay Module

Parts needed: Latching NC/NO E-Stop mushroom button (40 mm), 2-channel 5 V optocoupler relay module (JD-VCC isolated type).

The E-Stop is a **purely hardware circuit** — the relay is driven directly by the physical button's NC contact. No RPi GPIO pin is required. This is the correct fail-safe design: software cannot prevent or delay the ENABLE cut.

> ***Fail-safe principle:*** *When the E-Stop NC contact breaks (button pressed), the relay de-energises, NC contacts open, and ClearPath ENABLE drops to 0 V — even if the RPi is frozen or unpowered.*

How It Works

```
Most common relay modules are active-LOW: IN1 LOW = relay energised = NC closed.
Normal operation (E-Stop NOT pressed): NC contact closed → IN1 pulled to GND →
```

```
relay ON → NC contacts CLOSED → 5 V reaches ClearPath ENABLE → servos can run
E-Stop PRESSED (mushroom latched in): NC contact opens → 10 kΩ pulls IN1 HIGH →
relay OFF → NC contacts OPEN → ENABLE line cut → both ClearPath servos
immediately stop
```

Wiring

```
2-Ch Relay Module → Destination
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
VCC (coil supply) ■■■■ 5 V (buck module) GND ■■■■ Common GND JD-VCC (isolated)
■■■■ 5 V (buck module) ← keep isolated from VCC IN1 (CH1 trigger) ■■■■ E-Stop NC
circuit (see schematic below) IN2 (CH2 trigger) ■■■■ Tied to IN1 (both channels
trip together) COM1 ■■■■ 5 V (buck module) ← ENABLE supply rail NC1 ■■■■
ClearPath Z ENABLE (→ Level Converter B3) COM2 ■■■■ 5 V (buck module) NC2 ■■■■
ClearPath X ENABLE (→ Level Converter B3)
```

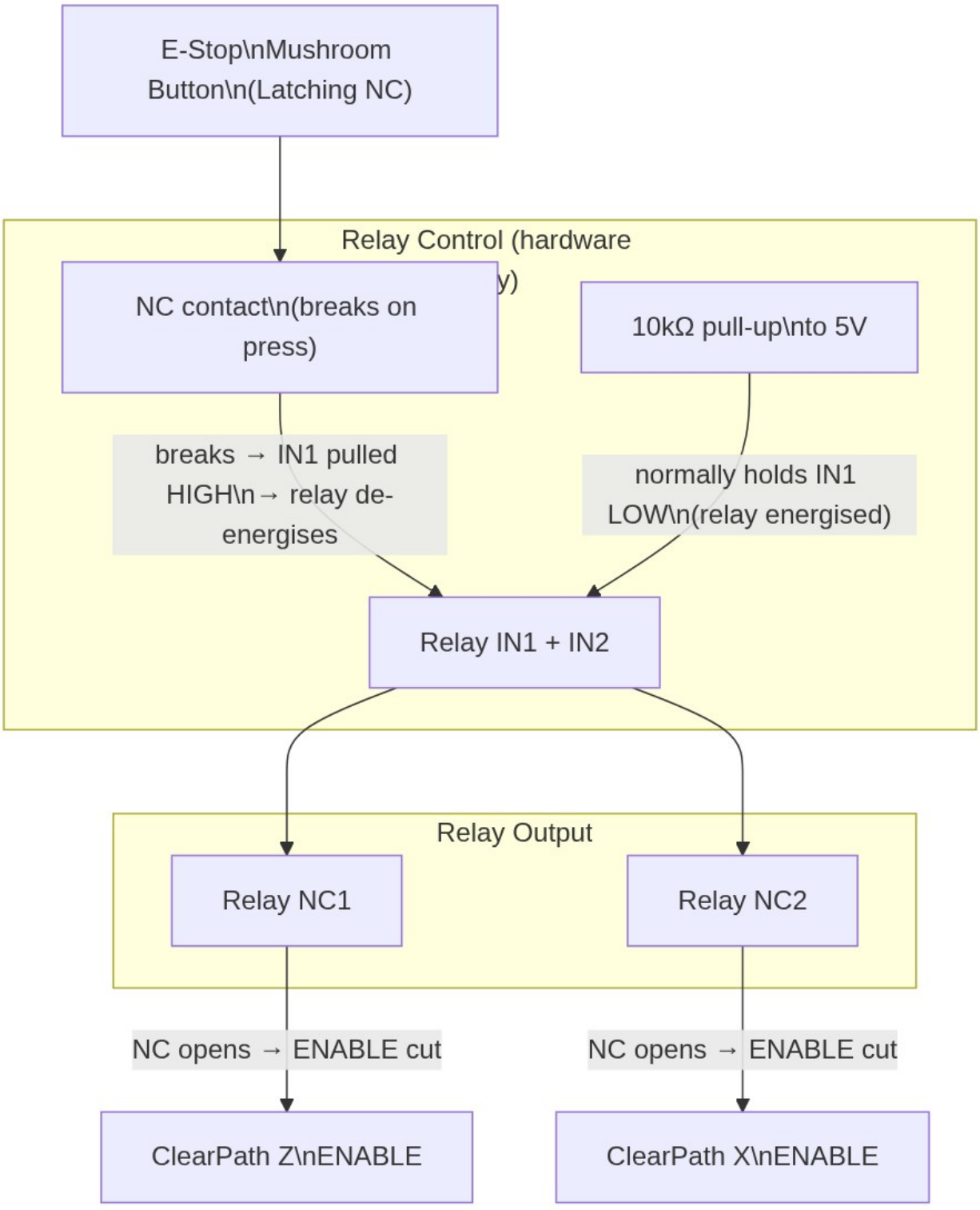
E-Stop Button → Relay Trigger Circuit

```
5 V (buck) ■■■■ 10 kΩ ■■■■■■■■■ Relay IN1 / IN2 ■ E-Stop NC contact ■ GND NC
closed (not pressed): IN1 = 0 V → relay ON → NC contacts CLOSED → ENABLE = 5 V NC
open (pressed): IN1 = 5 V → relay OFF → NC contacts OPEN → ENABLE = 0 V
```

> ■ *If your relay module is **active-HIGH** (less common), swap the logic: connect 5V through NC to IN1, and use a pull-down to GND instead.*

Complete E-Stop Schematic

```
Buck 5V ■■■■■ 10 kΩ ■■■■ IN1 ■■■ ■ ■ (active-LOW trigger) ■ E-Stop NC contact ■
■ ■ GND ■ ■■■■ COM1 ■■■■ Relay NC1 ■■■■ ClearPath Z ENABLE ■■■■ Level
Converter B3 ■■■■ COM2 ■■■■ Relay NC2 ■■■■ ClearPath X ENABLE ■■■■ Level
Converter B3
```



Verification — Step 13

Test	Expected Result	Tool
E-Stop NOT pressed — ENABLE voltage at ClearPath	~5 V (relay NC closed)	Multimeter

E-Stop PRESSED — ENABLE voltage at ClearPath	0 V (relay NC open)	Multimeter
Relay click audible on press	Yes — both channels	Audible
ClearPath HLFB LED on ENABLE cut	Amber / off (servo disabled)	Visual

---

5. Final System Verification

5.1 Pre-Power Checklist

- [ ] All GNDs tied to common ground bus
- [ ] Buck module output verified at 5.00 V before connecting any load
- [ ] No exposed wire ends contacting adjacent conductors
- [ ] Level shifter VCCA = 3.3 V, VCCB = 5 V verified (Steps 6 and 7)
- [ ] Spindle voltage divider output verified at ≤3.36 V (Step 5)
- [ ] ClearPath motors mechanically secured before applying 70 V
- [ ] E-Stop relay tested and ENABLE line confirmed cut on press (Step 13)

5.2 Full GPIO State Scan

```
# Run before starting the lathe application — confirms all pins are reachable
import pigpio PIN_MAP = { "Z Enc A": (5, "IN", pigpio.PUD_UP), "Z Enc B": (6, "IN",
pigpio.PUD_UP), "X Enc A": (13, "IN", pigpio.PUD_UP), "X Enc B": (19, "IN",
pigpio.PUD_UP), "Spindle": (12, "IN", pigpio.PUD_DOWN), "Z STEP": (17, "OUT",
None), "Z DIR": (27, "OUT", None), "Z ENABLE": (22, "OUT", None), "X STEP": (23,
"OUT", None), "X DIR": (24, "OUT", None), "X ENABLE": (25, "OUT", None), # Note:
E-Stop relay is purely hardware-driven (button NC → relay IN1). # No RPi GPIO pin
is assigned to E-Stop in config.py. "Button 1": (26, "IN", pigpio.PUD_UP), "Button
2": (20, "IN", pigpio.PUD_UP), "Button 3": (21, "IN", pigpio.PUD_UP), "Half-Nut":
(4, "IN", pigpio.PUD_DOWN), "Limit Z+": (16, "IN", pigpio.PUD_UP), "Limit Z-": (7,
"IN", pigpio.PUD_UP), "Limit X+": (8, "IN", pigpio.PUD_UP), "Limit X-": (11, "IN",
pigpio.PUD_UP), } pi = pigpio.pi() print(f"{'Signal':<14} {'GPIO':>5} {'Level'}")
print("-" * 32) for name, (pin, direction, pud) in PIN_MAP.items(): if direction ==
"IN": pi.set_mode(pin, pigpio.INPUT) pi.set_pull_up_down(pin, pud) level =
pi.read(pin) print(f"{name:<14} GPIO {pin:<3} {level}") else: pi.set_mode(pin,
pigpio.OUTPUT) pi.write(pin, 0) print(f"{name:<14} GPIO {pin:<3} [OUTPUT]")
pi.stop()
```

5.3 I2C Bus Scan

```
i2cdetect -y 1 # Expected: address 0x48 (ADS1115) is present on the bus
```

5.4 First Motion Test (Low-Speed Jog)

Complete all above steps and verifications first, then:

1. Connect a multimeter to ClearPath Z ENABLE line — confirm it reads 5 V (relay closed, E-Stop not pressed).
2. Launch the application: `python3 main.py --windowed`
3. Keep E-Stop button within arm's reach at all times.
4. Enable Z servo: software asserts GPIO 22 HIGH (via level shifter → ClearPath ENABLE HIGH).
5. Rotate Z handwheel slowly — Z carriage should track smoothly.
6. Test E-Stop: press the mushroom button — motor must stop immediately and ENABLE voltage drops to 0 V.
7. Release E-Stop (twist to unlatch) and repeat for X axis.

GPIO Quick Reference

```
RPi 40-pin Header — BCM Pin Map (this project)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Phys ■
BCM ■ Signal ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ 1 ■ 3.3
V ■ Power — Encoders, ADS1115 ■ ■ 2 ■ 5 V ■ Power — Level Shifter VCCB ■ ■ 3 ■
GPIO 2 (SDA) ■ I2C SDA → ADS1115 ■ ■ 4 ■ 5 V ■ Power — Level Shifter VCCB ■ ■ 5 ■
GPIO 3 (SCL) ■ I2C SCL → ADS1115 ■ ■ 6 ■ GND ■ Ground ■ ■ 7 ■ GPIO 4 ■ Half-Nut
lever switch ■ ■ 11 ■ GPIO 17 ■ Z STEP → LS-Z A1 ■ ■ 13 ■ GPIO 27 ■ Z DIR → LS-Z
A2 ■ ■ 15 ■ GPIO 22 ■ Z ENABLE→ LS-Z A3 ■ ■ 16 ■ GPIO 23 ■ X STEP → LS-X A1 ■ ■
17 ■ 3.3 V ■ Power — Encoders (2nd tap) ■ ■ 18 ■ GPIO 24 ■ X DIR → LS-X A2 ■ ■ 22
■ GPIO 25 ■ X ENABLE→ LS-X A3 ■ ■ 23 ■ GPIO 11 ■ Limit X– ■ ■ 24 ■ GPIO 8 ■
Limit X+ ■ ■ 26 ■ GPIO 7 ■ Limit Z– ■ ■ 29 ■ GPIO 5 ■ Z Encoder A ■ ■ 31 ■ GPIO 6
■ Z Encoder B ■ ■ 32 ■ GPIO 12 ■ Spindle index (via V-div) ■ ■ 33 ■ GPIO 13 ■ X
Encoder A ■ ■ 35 ■ GPIO 19 ■ X Encoder B ■ ■ 36 ■ GPIO 16 ■ Limit Z+ ■ ■ 37 ■
GPIO 26 ■ Button 1 (X memory) ■ ■ 38 ■ GPIO 20 ■ Button 2 (Z memory) ■ ■ 40 ■
GPIO 21 ■ Button 3 (Units / stop) ■ ■ 6/9 ■ GND ■ Ground (use all GND pins) ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ LS-Z =
TXS0108E Level Shifter (Z axis) LS-X = TXS0108E Level Shifter (X axis)
```